



Python Programming

Ch-3 Python Functions, Modules and Packages

Course Outcome : Develop python program using Function, modules and packages.

Prof. Perna S. Patil
Department of Computer Engineering
School of Polytechnic & Skill Development

Points to be Covered

Python Functions ,Modules and Packages:

3.1 Functions – Defining Functions, Calling Functions, Passing Arguments, Keyword Arguments, Default Arguments, Variable-length arguments, Scope of the Variables in a Function- Global and Local Variables , Use of Python Built-in Function

3.2 Modules: Creating modules, import statements, Importing module, Math module, Random module, namespacing and Scoping .

3.3 Packages: Python packages, Introduction to PIP, Installing Packages via PIP, Using Standard (e.g.math,scipy,Numpy, matplotlib, panda,etc) and User defined Packages.

Python Functions ,Modules and Packages:



SINCE 1983

Dr. Vishwanath Karad
**MIT WORLD PEACE
UNIVERSITY** | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS



Function

A function is a block of code which only runs when it is called.

You can pass data, known as parameters, into a function.

A function can return data as a result.

Creating a Function

In Python a function is defined using the def keyword:

Example

```
def my_function():  
    print("Hello from a function")
```

Arguments

Information can be passed into functions as arguments.

Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

Python Functions ,Modules and Packages.

Calling a Function

Defining a function only gives it a name, specifies the parameters that are to be included in the function and structures the blocks of code.

Once the basic structure of a function is finalized, you can execute it by calling it from another function or directly from the Python prompt. Following is the example to call printme() function –

Function definition is here

```
def printme( str ):  
    "This prints a passed string into this function"  
    print str  
    return;
```

Now you can call printme function

```
printme("I'm first call to user defined function!")  
printme("Again second call to the same function")
```

Python Functions ,Modules and Packages:



SINCE 1983

Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Supported by



UNESCO Chair for
Human Rights, Democracy, Peace & Tolerance
World Peace Centre (Alandi) Pune, India

Passing Arguments

Pass by reference vs value

All parameters (arguments) in the Python language are passed by reference. It means if you change what a parameter refers to within a function, the change also reflects back in the calling function. For example –

```
def changeme( mylist ):
```

```
    "This changes a passed list into this function"
```

```
    mylist.append([1,2,3,4]);
```

```
    print "Values inside the function: ", mylist
```

```
    return
```

```
# Now you can call changeme function
```

```
mylist = [10,20,30];
```

```
changeme( mylist );
```

```
print "Values outside the function: ", mylist
```

Python Functions ,Modules and Packages:

Pass By Reference:

- Python passes parameters to a function using a technique known as pass by reference. This means that when we pass parameters, the function refers to the original passed values using new names.
- For example, consider the following simple program:

```
# reference.py
```

```
def add(a, b):
```

```
    return a + b
```

- Run IDLE's interactive command line and type following:

```
>>> x, y = 3, 4
```

```
>>> add(x, y)
```

2. Pass By Value:

- In pass by value, the function receives a copy of the argument objects passed to it by the caller, stored in a new location in memory.
- When a parameter is passed by value, a copy of it is made and passed to the function. If the value being passed is large, the copying can take up a lot of time and memory. Python, however, does not support pass by value.

Python Functions ,Modules and Packages:

Function Arguments

You can call a function by using the following types of formal arguments –

Required arguments

Keyword arguments

Default arguments

Variable-length arguments

Required arguments

Required arguments are the arguments passed to a function in correct positional order. Here, the number of arguments in the function call should match exactly with the function definition.

To call the function *printme()*, you definitely need to pass one argument, otherwise it gives a syntax error as follows –

Python Functions ,Modules and Packages:

```
# Function definition is here
def printme( str ):
    "This prints a passed string into this function"
    print str
    return;
```

```
# Now you can call printme function
printme()
```

When the above code is executed, it produces the following result –

```
Traceback (most recent call last):
  File "test.py", line 11, in <module>
    printme();
TypeError: printme() takes exactly 1 argument (0 given)
```

Python Functions ,Modules and Packages:

In-Built Mathematical Functions

Various mathematical functions in Python programming are explained below:

1. abs(): Return the absolute value of a number.

Syntax: abs(number)

Example: For abs() function.

```
>>> abs(10)
```

```
10
```

```
>>> abs(-10)
```

```
10
```

Python Functions ,Modules and Packages:

Keywords Arguments

- Keyword arguments are related to the function calls. When we use keyword arguments in a function call, the caller identifies the arguments by the parameter name.
- This allows us to skip arguments or place them out of order because the Python interpreter is able to use the keywords provided to match the values with parameters.

Example 1: For keywords arguments.

```
>>> def display(str):  
    "This print a string passed as argument"  
    print(str)  
    return  
>>> display()  
Traceback (most recent call last):  
  File "<pyshell#17>", line 1, in <module>  
    display()  
TypeError: display() missing 1 required positional argument: 'str'  
>>> display(str="Hello Python")  
Hello Python
```

Ex-2

```
def display(name,age):  
    print("Name:",name)  
    print("Age:",age)  
    return  
>>> display(name="meenakshi",age=35)  
Name: meenakshi  
Age: 35  
>>> display(age=35,name="meenakshi")  
Name: meenakshi  
Age: 35
```

Python Functions ,Modules and Packages:

2. Default Arguments

· A default argument is an argument that assumes a default value if a value is not provided in the function call for that argument.

Example: For default arguments.

```
>>> def display(name,age=20):  
print("Name:",name)  
print("Age:",age)  
return  
>>> display(name="meenakshi")  
Name: meenakshi  
Age: 20
```

Variable Length Arguments

In many cases where we are required to process a function with more number of arguments than we specified in the function definition. These types of arguments are known as variable length arguments.

Syntax:

```
def function-name([formal_args,] *var_args_tuple ):  
"function_docstring"  
function_statements  
return [expression]
```

An asterisk (*) is placed before the variable name that holds the values of all nonkeyword variable arguments. This tuple remains empty if no additional arguments are specified during the function call

Python Functions ,Modules and Packages:

For variable length argument.

```
>>> def display(arg1,*list):  
    "This prints a variable passed arguments"  
    print(arg1)  
    for i in list:  
        print(i)  
    return  
>>> display(10,20,30)  
10  
20  
30  
>>>>>>
```

Python Functions ,Modules and Packages:

Scope of Variable

- All variables in a program may not be accessible at all locations in that program. This depends on where we have declared a variable.
- The scope of a variable determines the portion of the program where you can access a particular variable/identifier.
- The availability/accessibility of a variable in different parts of a program is referred to as its scope.

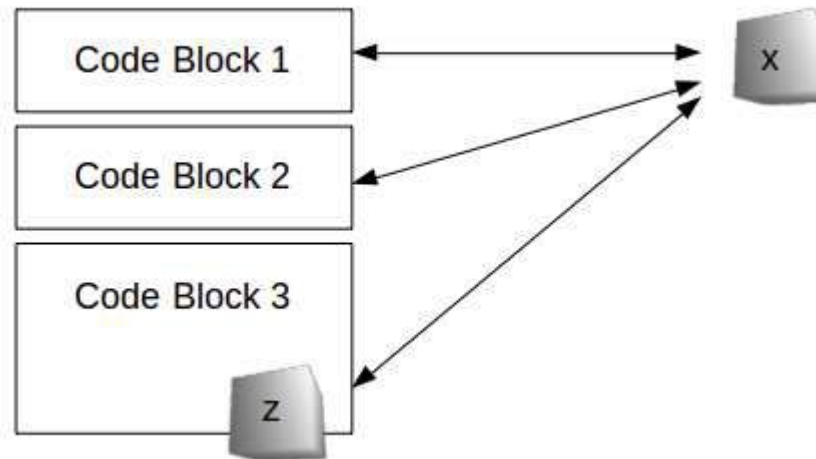
There are two basic scopes of variables in Python:

1. Global Variables: Global variables can be accessed throughout (outside) the program body by all functions.

2. Local Variables: Local variables can be accessed only inside the function in which they are declared.

For scope of variables.

```
>>> g=10 # global variable g
>>> def show():
l=20 # local variable l
print("local variable=",l)
print("Global variable=",g)
>>> show()
local variable= 20
Global variable= 10
```



Python Functions ,Modules and Packages:



SINCE 1983
Dr. Vishwanath Karad
**MIT WORLD PEACE
UNIVERSITY** | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS



MODULES

Modules are primarily the (.py) files which contain Python programming code defining functions, class, variables, etc. with a suffix .py appended in its file name.

A file containing .py python code is called a module.

In Python we can put definitions in a file and use them in a script or in an interactive instance of the interpreter. Such a file is called a module

Writing Module

- Writing a module means simply creating a file which can contains python definitions and statements. The file name is the module name with the extension .py. To include module in a file, use import statement.

Python Functions ,Modules and Packages:

Follow the following steps to create modules:

1. Create a first file as a python program with extension as .py. This is your module file where we can write a function which perform some task.
2. Create a second file in the same directory called main file where we can import the module to the top of the file and call the function.
3. Second file needs to be in the same directory so that Python knows where to find the module since it's not a built-in module.

Example : For creating a module. Type the following code and save it as p1.py.

```
def add(a, b):
```

```
"This function adds two numbers and return the result"
```

```
result = a + b
```

```
return result
```

```
def sub(a, b):
```

```
"This function subtract two numbers and return the result"
```

```
result = a - b
```

```
return result
```

Python Functions ,Modules and Packages:

Importing Modules

- Import statement is used to imports a specific module by using its name. Import statement creates a reference to that module in the current namespace. After using import statement we can use refer to things defined in that module.
- We can import the definitions inside a module to another module or the interactive interpreter in Python. We use the import keyword to do this.

□ Import the definitions inside a module:

```
import p1  
print(p1.add(10,20))  
print(p1.sub(20,10))
```

Output:

```
30  
10
```

· Import the definitions using the interactive interpreter:

```
>>> import p1  
>>> p1.add(10,20)  
30  
>>> p1.sub(20,10)  
10
```

Python Functions ,Modules and Packages:

Importing Modules

- Import statement is used to imports a specific module by using its name. Import statement creates a reference to that module in the current namespace. After using import statement we can use refer to things defined in that module.
- We can import the definitions inside a module to another module or the interactive interpreter in Python. We use the import keyword to do this.

· **Import the definitions inside a module:**

```
import p1  
print(p1.add(10,20))  
print(p1.sub(20,10))
```

Output:

```
30  
10
```

· **Import the definitions using the interactive interpreter:**

```
>>> import p1  
>>> p1.add(10,20)  
30  
>>> p1.sub(20,10)  
10  
>>>
```

Python Functions ,Modules and Packages:

Importing Objects From Module:

· Import in python is similar to `#include header_file` in C/C++. Python modules can get access to code from another module by importing the file/function using `import`. Python provides three different ways to import modules.

1. From x import a:

- Imports the module x, and creates references in the current namespace to all public objects defined by that module. If we run this statement, we can simply use a plain name to refer to things defined in module x.
- We can access pi directly without dot notation.

Example: For importing module.

```
>>> from math import pi
>>> pi
3.141592653589793
>>> from math import sqrt
>>> sqrt(144)
12.0
```

Python Functions ,Modules and Packages:

Python Built in Modules

A module is a collection of Python objects such as functions, classes, and so on. Python interpreter is bundled with a standard library consisting of large number of built-in modules,

- Built-in modules are generally written in C and bundled with Python interpreter in precompiled form. A built-in module may be a Python script (with .py extension) containing useful utilities.
- A module may contain one or more functions, classes, variables, constants, or any other Python resources.

Numeric and Mathematical Modules:

· This module provides numeric and math-related functions and data types. Following are the modules which are classified as numeric and mathematical modules

- (i) numbers (Numeric abstract base classes).
- (ii) math (Mathematical functions).
- (iii) cmath (Mathematical functions for complex numbers).
- (iv) decimal (Decimal fixed point and floating point arithmetic).
- (v) fractions (Rational numbers).
- (vi) random (Generate pseudo-random numbers).
- (vii) statistics (Mathematical statistics functions).

Python Functions ,Modules and Packages:

Random Module:

- Sometimes, we want the computer to pick a random number in a given range, pick a random element from a list etc.
- The random module provides functions to perform these types of operations. This function is not accessible directly, so we need to import random module and then we need to call this function using random static object.

· **Example: For random module.**

```
>>> import random
```

```
>>> print(random.random()) # It generate a random number in the range (0.0, 1.0)  
0.27958089234907935
```

```
>>> print(random.randint(10,20)) # It generate a random integer between x and y inclusive  
13
```

Python Functions ,Modules and Packages:



SINCE 1983
Dr. Vishwanath Karad
**MIT WORLD PEACE
UNIVERSITY** | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS



Namespace and Scoping

A namespace is a system to have a unique name for each and every object in Python. An object might be a variable or a method.

Python itself maintains a namespace in the form of a Python dictionary

A namespace in python is a collection of names. So, a namespace is essentially a mapping of names to corresponding objects.

- At any instant, different python namespaces can coexist completely isolated- the isolation ensures that there are no name collisions/problem.
- A scope refers to a region of a program where a namespace can be directly accessed, i.e. without using a namespace prefix.
- Scoping in Python revolves around the concept of namespaces. Namespaces are basically dictionaries containing the names and values of the objects within a given scope

Python Functions ,Modules and Packages:



SINCE 1983
Dr. Vishwanath Karad
**MIT WORLD PEACE
UNIVERSITY** | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS



Types of Namespaces:

- When a user creates a module, a global namespace gets created, later creation of local functions creates the local namespace. The built-in namespace encompasses global namespace and global namespace encompasses local namespace.

1. Local Namespace: This namespace covers the local names inside a function. Python creates this namespace for every function called in a program. It remains active until the function returns.

2. Global Namespace: This namespace covers the names from various imported modules used in a project. Python creates this namespace for every module included in your program. It will last until the program ends.

3. Built-in Namespace: This namespace covers the built-in functions and built-in exception names. Python creates it as the interpreter starts and keeps it until we exit.

Python Functions ,Modules and Packages:

Example: For global and local scope.

```
global_var = 30 # global scope
```

```
def scope():
```

```
    local_var = 40 # local scope
```

```
    print(global_var)
```

```
    print(local_var)
```

```
    scope()
```

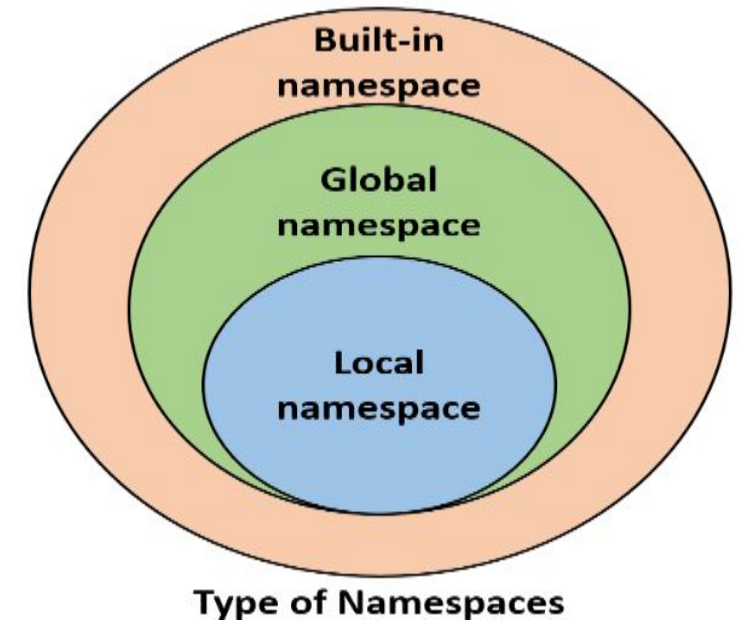
```
    print(global_var)
```

Output:

30

40

3



Python Functions ,Modules and Packages:

PYTHON PACKAGES

- Suppose we have developed a very large application that includes many modules. As the number of modules grows, it becomes difficult to keep track of them all.
- This is particularly so if they have similar names or functionality. It is necessary to group and organize them by some mean which can be achieved by packages.

Introduction

- A package is a hierarchical file directory structure that defines a single Python application environment that consists of modules and subpackages and sub-subpackages and so on.
- Packages allow for a hierarchical structuring of the module namespace using dot notation.

Packages

are a way of structuring many packages and modules which help in a well-organized hierarchy of

data set, making the directories and modules easy to access.

- A package is a collection of Python modules, i.e., a package is a directory of Python modules

containing an additional `__init__.py` file (For example: Phone/`__init__.py`).

Python Functions ,Modules and Packages:

Here, there is a directory named mypkg that contains two modules, p1.py and p2.py. The contents of the modules are:

p1.py

```
def m1():  
    print("first module")
```

p2.py

```
def m2():  
    print("second module")
```

· Here mypkg a folder /directory which consist of p1.py and p2.py.we can refer these two modules with dot notation(mypkg.p1, mypkg.p2) and import them with the one of the following syntaxes:

Syntax 1:

```
import <module_name>[, <module_name> ...]
```

Python Functions ,Modules and Packages:

Example:

```
>>> import mypkg.p1, mypkg.p2
>>> mypkg.p1.m1()
first module
>>> p1.m1()
```

Syntax 2:

```
from <module_name> import <name(s)>
```

Example:

```
>>> from mypkg.p1 import m1
>>> m1()
first module
>>>
```

Syntax 3:

```
from <module_name> import <name> as <alt_name>
```

Example:

```
>>> from mypkg.p1 import m1 as function
>>> function()
first module
>>>
```

Syntax 4:

```
from <package_name> import
<modules_name>[, <module_name>
...]
```

Example:

```
>>> from mypkg import p1, p2
>>> p1.m1()
first module
>>> p2.m2()
second module
>>>
```

Python Functions ,Modules and Packages:



Introduction to PIP

pip is the standard package manager for Python. It allows you to install and manage additional packages that are not part of the Python standard library.

The Python installer installs pip, so it should be ready for you to use, unless you installed an old version of Python. You can verify that pip is available by running the following command in your console:

```
$ pip --version (shell)
```

How to install pip?

pip comes pre-installed on the Python versions 3.4 or older. We can check if pip is installed by using the following command in the console:

```
pip --version
```

If pip is already available in the system, the respective pip version is displayed, like:

```
pip 19.3.1 from C:\Python37\lib\site-packages\pip (python 3.7)
```

pip install numpy

If we had installed pip on our system, this command would have installed the numpy library.

Python Functions ,Modules and Packages:

Standard Packages

- NumPy and SciPy are the standards packages used by Python programming.
- NumPy enriches the programming language Python with powerful data structures, implementing multi-dimensional arrays and matrices.
- SciPy (Scientific Python) is often mentioned in the same breath with NumPy. SciPy needs Numpy, as it is based on the data structures of Numpy and furthermore its basic creation and manipulation functions.

Math

- Some of the most popular mathematical functions are defined in the math module. These include trigonometric functions, representation functions, logarithmic functions and angle conversion functions.
- Two mathematical constants are also defined in this module.
- Pie (π) is a well-known mathematical constant, which is defined as the ratio of the circumference to the diameter of a circle and its value is 3.141592653589793.

```
>>> import math
```

```
>>> math.pi
```

```
3.141592653589793
```

Python Functions ,Modules and Packages:

Import NumPy

Once NumPy is installed, import it in your applications by adding the import keyword:
import numpy ,Now NumPy is imported and ready to use.

Example

```
import numpy  
arr = numpy.array([1, 2, 3, 4, 5])  
print(arr)
```

Using NumPy, a developer can perform the following operations:

1. Mathematical and logical operations on arrays.
2. Fourier transforms and routines for shape manipulation.
3. Operations related to linear algebra.
4. NumPy has in-built functions for linear algebra and random number generation.

NumPy as np

NumPy is usually imported under the np alias.

alias: In Python alias are an alternate name for referring to the same thing.

Create an alias with the as keyword while importing:

```
import numpy as np
```

Example

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5])
```

```
print(arr)
```

Python Functions ,Modules and Packages:

NumPy

- NumPy is the fundamental package for scientific computing with Python. NumPy stands for "Numerical Python". It provides a high-performance multidimensional array object, and tools for working with these arrays.
- An array is a table of elements (usually numbers), all of the same type, indexed by a tuple of positive integers and represented by a single variable.

NumPy's array class is called ndarray. It is also known by the alias array.

Array Object:

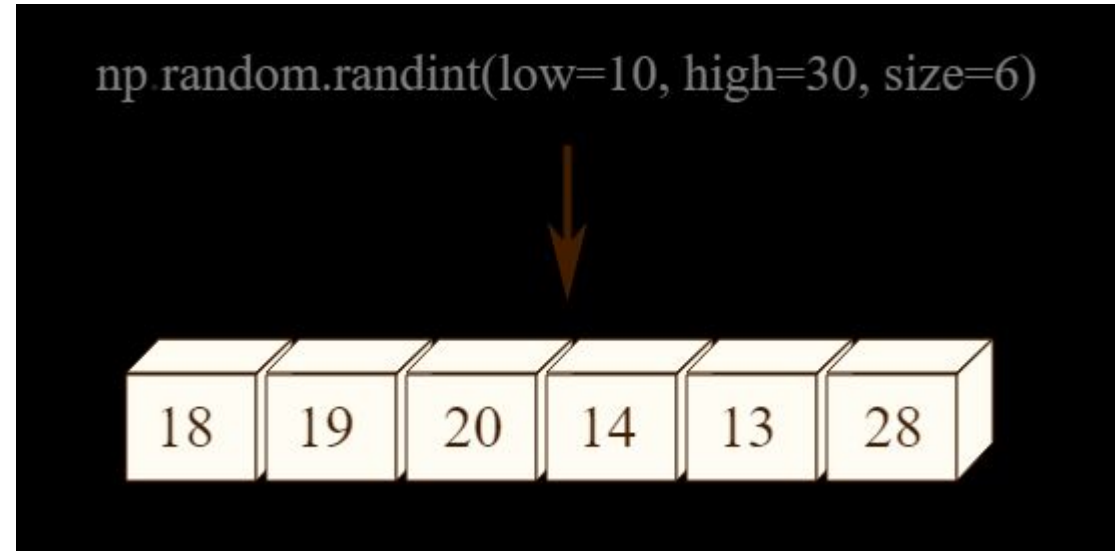
- NumPy's main object is the homogeneous multidimensional array. It is a table of elements (usually numbers), all of the same type, indexed by a tuple of positive integers.
- In NumPy dimensions are called axes. The number of axes is rank. NumPy's array class is called ndarray. It is also known by the alias array.
- Basic attributes of the ndarray class as follow:

Sr. No.	Attributes	Description
1.	Shape	A tuple that specifies the number of elements for each dimension of the array.
2.	Size	The total number elements in the array.
3.	Ndim	Determines the dimension an array.
4.	nbytes	Number of bytes used to store the data.
5.	dtype	Determines the datatype of elements stored in array.

Python Functions ,Modules and Packages:

An array of **random integers** can be generated using the randint() **NumPy** function.
This function takes three arguments, the lower end of the range, the upper end of the range, and the number of **integer** values to **generate** or the size of the array

```
import numpy as np  
x = np.random.randint(low=10, high=30, size=6)  
print(x)
```

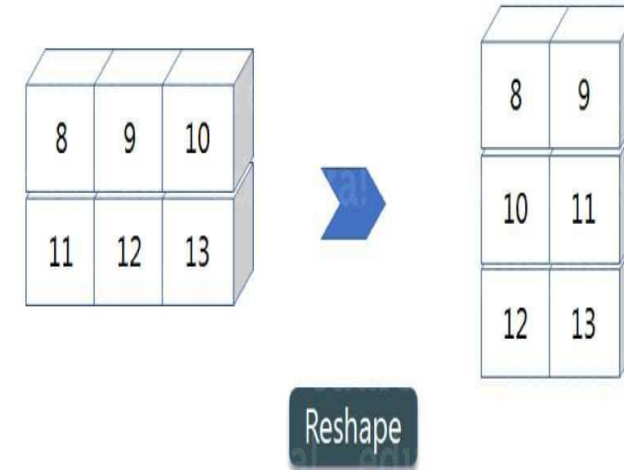


Python Functions ,Modules and Packages:

We can also perform reshape operation using python numpy operation. Reshape is when you change the number of rows and columns which gives a new view to an object.

Example:

```
>>> arr=np.array([[1,2,3],[4,5,6]])  
>>> a=arr.reshape(3,2)  
>>> a  
array([[1, 2],  
       [3, 4],  
       [5, 6]])
```



Python Functions ,Modules and Packages:

SciPy

SciPy is a library that uses NumPy for more mathematical functions. SciPy uses NumPy arrays as the basic data structure, and comes with modules for various commonly used tasks in scientific programming, including linear algebra, integration (calculus), ordinary differential equation solving, and signal processing.

- Following command execute to install SciPy in Window, Linux and MAC OS:

```
python -m pip install scipy
```

- SciPy is organized into subpackages covering different scientific computing domains. These are summarized in the following table:

Sr. No.	Subpackage	Description
1.	cluster	Clustering algorithms.
2.	constants	Physical and mathematical constants.
3.	fftpack	Fast Fourier Transform routines.
4.	integrate	Integration and ordinary differential equation solvers.
5.	interpolate	Interpolation and smoothing splines.
6.	io	SciPy has many modules, classes, and functions available read data from and write data to a variety of file formats.
7.	linalg	Linear algebra.
8.	ndimage	N-dimensional image processing.
9.	odr	Orthogonal distance regression.
10.	optimize	Optimization and root-finding routines.
11.	signal	Signal processing.
12.	sparse	Sparse matrices and associated routines.
13.	spatial	Spatial data structures and algorithms.
14.	special	Special functions.
15.	stats	Statistical distributions and functions.

Python Functions ,Modules and Packages:

Matplotlib

matplotlib.pyplot is a plotting library used for 2D graphics in python programming language. It can be used in python scripts, shell, web application servers and other graphical user interface toolkits.

There are various plots which can be created using python matplotlib like bar graph, histogram, scatter plot, area plot, pie plot.

- Following command execute to install matplotlib in Window, Linux and MAC OS:

```
python -m pip install matplotlib
```

Importing matplotlib:

```
from matplotlib import pyplot as plt
```

OR

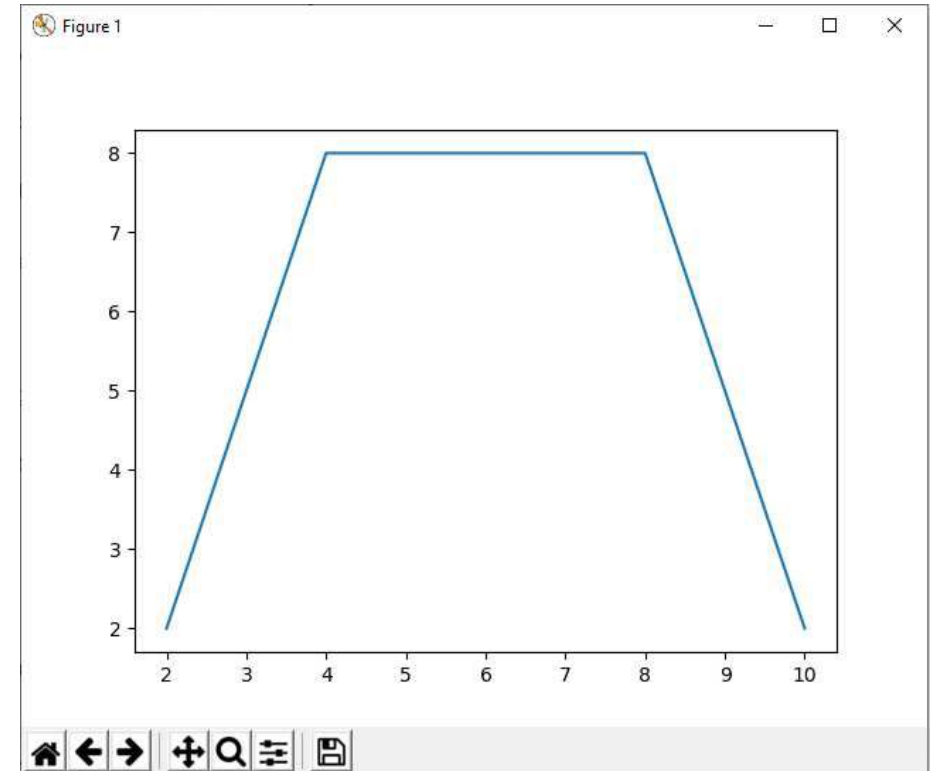
```
import matplotlib.pyplot as plt
```

Python Functions ,Modules and Packages:

Example: Line Plot

```
>>> from matplotlib import pyplot as plt  
>>> x=[2,4,8,10]  
>>> y=[2,8,8,2]  
>>> plt.plot(x,y)  
[<matplotlib.lines.Line2D object at  
0x02E69B70>]  
>>> plt.show()
```

Output



Python Functions ,Modules and Packages:

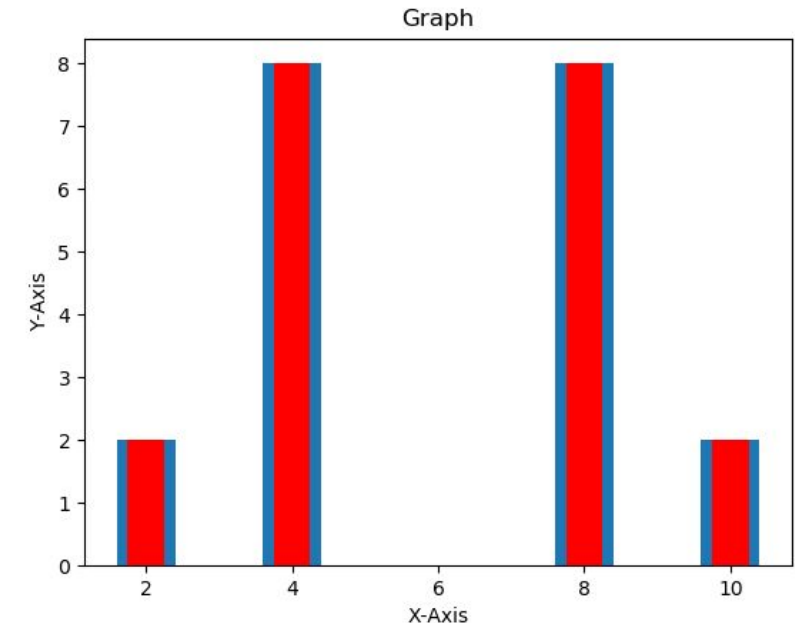
Bar Graph:

· A bar graph uses bars to compare data among different categories. It is well suited when you want to measure the changes over a period of time. It can be represented horizontally or vertically.

Example: For bar graph.

```
>>> from matplotlib import pyplot as plt
>>> x=[2,4,8,10]
>>> y=[2,8,8,2]
>>> plt.xlabel('X-Axis')
Text(0.5, 0, 'X-Axis')
>>> plt.ylabel('Y-Axis')
Text(0, 0.5, 'Y-Axis')
>>> plt.title('Graph')
Text(0.5, 1.0, 'Graph')
>>> plt.bar(x,y,label="Graph",color='r',width=.5)
<BarContainer object of 4 artists>
>>> plt.show()
```

Output: -----?

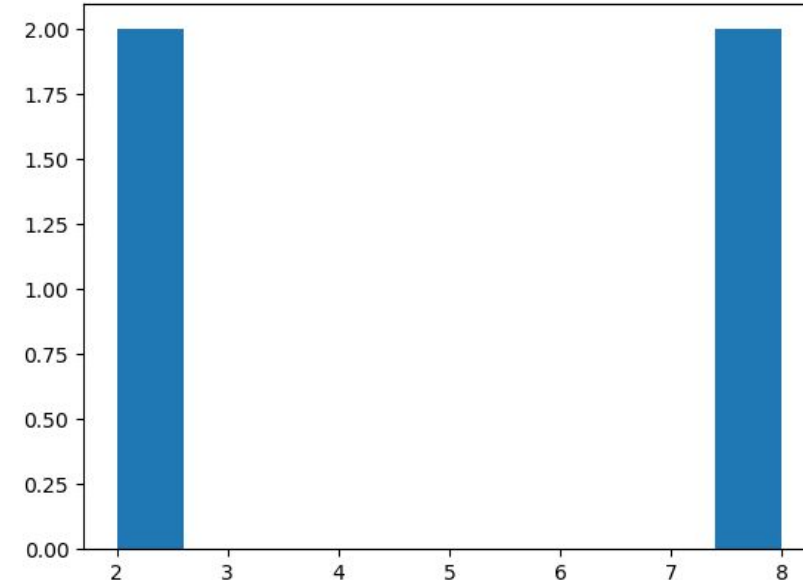


Python Functions ,Modules and Packages:

For histogram.

```
>>> from matplotlib import pyplot as plt  
>>> y=[2,8,8,2]  
>>> plt.hist(y)  
(array([2., 0., 0., 0., 0., 0., 0., 0., 0., 2.]),  
 array([2. , 2.6, 3.2, 3.8, 4.4, 5.  
 , 5.6, 6.2, 6.8, 7.4, 8. ]), <a list of 10 Patch  
objects>)  
>>> plt.show()
```

Output:



Python Functions ,Modules and Packages:

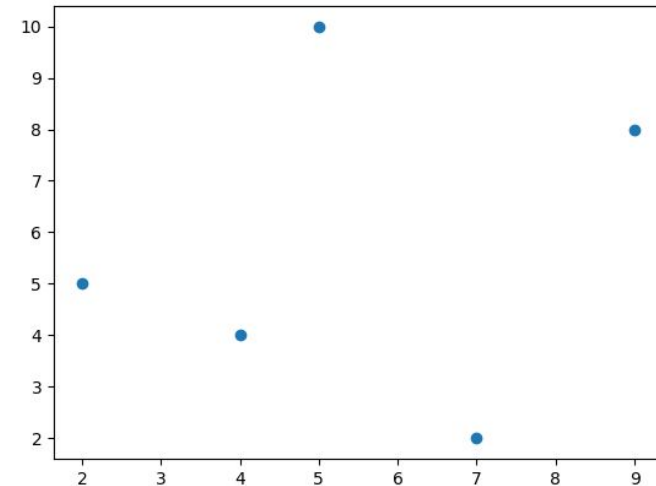
Scatter Plot:

- Usually we need scatter plots in order to compare variables, for example, how much one variable is affected by another variable to build a relation out of it.
- The data is displayed as a collection of points, each having the value of one variable which determines the position on the horizontal axis and the value of other variable determines the position on the vertical axis.

Example: For scatter plot.

```
>>> from matplotlib import pyplot as plt  
>>> x = [5, 2, 9, 4, 7]  
>>> y = [10, 5, 8, 4, 2]  
>>> plt.scatter(x,y)  
<matplotlib.collections.PathCollection object at 0x05792770>  
>>> plt.show()
```

Output:



Python Functions ,Modules and Packages:

Pandas

- Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures.
- It is built on the Numpy package and its key data structure is called the DataFrame. DataFrames allow you to store and manipulate tabular data in rows of observations and columns of variables.

Installing Pandas:

pip install pandas

Data structures supported by Pandas:

- Pandas deals with the following three data structures:

Sr. No.	Data Structure	Dimensions	Description
1.	Series	1	1D labeled homogeneous array, size immutable.
2.	Data Frames	2	General 2D labeled, size-mutable tabular structure with potentially heterogeneously typed columns.
3.	Panel	3	General 3D labeled, size-mutable array.

Series:

Python Functions ,Modules and Packages:

- Series is a one-dimensional array like structure with homogeneous data. The Series is a one Dimensional array which is Labelled and it is capable of holding array of any type like Integer, Float, String and Python Objects.
- For example, the following series is a collection of integers 10, 22, 30, 40,...
`pandas.Series(data, index, dtype, copy)`
- **It takes four arguments:**
 - 1. data:** It is the array that needs to be passed so as to convert it into a series. This can be Python lists, NumPy Array or a Python Dictionary or Constants.
 - 2. index:** This holds the index values for each element passed in data. If it is not specified, default is `numpy.arange(length_of_data)`.
 - 3. dtype:** It is the datatype of the data passed in the method.
 - 4. copy:** It takes a Boolean value specifying whether or not to copy the data. If not specified, default is false.

Python Functions ,Modules and Packages:

Example 1: Using Series data structure of Panda.

```
>>> import pandas as pd
>>> import numpy as np
>>> numpy_arr = array([2, 4, 6, 8, 10,
20])
>>> si = pd.Series(arr)
>>> print(si)
0 2
1 4
2 6
3 8
4 10
5 20
dtype: int32
```

Example 2: Using Series data structure of Panda.

```
>>> import pandas as pd
>>> Data=[10,20,30,40,50]
>>> Index=['a','b','c','d','e']
>>> si=pd.Series(Index,Data)
>>> si
10 a
20 b
30 c
40 d
50 e
dtype: object
```

Python Functions ,Modules and Packages:

User Defined Packages

- We organize a large number of files in different folders and subfolders based on some criteria, so that we can find and manage them easily. In the same way, a package in Python takes the concept of the modular approach to next logical level.
- As we know, a module can contain multiple objects, such as classes, functions, etc. A package can contain one or more relevant modules.
- Physically, a package is actually a folder containing one or more module files. Let's create a package named MyPkg, using the following steps:

Step 1: Create a folder MyPkgon “C:\Users\Meenakshi\AppData\Local\Programs\Python\Python37\”. Create modules Message.py and Mathematics.py with following code:

Message.py

```
defSayHello(name):  
print("Hello " + name)  
return
```

Mathematics.py

```
def sum(x,y):  
returnx+y  
def average(x,y):  
return (x+y)/2  
def power(x,y):  
return x**y
```

Python Functions ,Modules and Packages:

Step 2: Create an empty `__init__.py` file in the MyPkg folder.

The package folder contains a special file called `__init__.py`, which stores the package's content.

It serves two purposes:

1. The Python interpreter recognizes a folder as the package if it contains `__init__.py` file.
 2. `__init__.py` exposes specified resources from its modules to be imported.
- An empty `__init__.py` file makes all functions from above modules available when this package is imported. Note that `__init__.py` is essential for the folder to be recognized by Python as a package.

We can optionally define functions from individual modules to be made available.

Step 3: Create P1.py file in MyPkg folder and write following code:

```
from MyPkg import Mathematics  
from MyPkg import Message  
greet.SayHello("Meenakshi")  
x = functions.power(3,2)  
print("power(3,2) : ", x)
```

Output:

```
Hello Meenakshi  
power(3,2) : 9
```

Python Functions ,Modules and Packages:

Let's create a package named mypackage, using the following steps:

Create a new folder named D:\MyApp.

Inside MyApp, create a subfolder with the name 'mypackage'.

Create an empty __init__.py file in the mypackage folder.

Using a Python-aware editor like IDLE, create modules greet.py and functions.py with following code:

<https://www.tutorialsteacher.com/python/python-package>

Python Functions ,Modules and Packages:



SINCE 1983
Dr. Vishwanath Karad
**MIT WORLD PEACE
UNIVERSITY** | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

